

Bokningssystem

Projektdokumentation

*En modern, responsiv plattform for gympassbokning,
byggd for Core Gym Club AB i Haninge.*

1. Projektöversikt

Bakgrund

Core Gym Club AB är en gymkedja i Haninge som erbjuder gruppträning, personlig träning och öppen gymtid. Företaget efterfrågade ett digitalt bokningssystem som gör det möjligt för medlemmar att boka gympass, se scheman och hantera sina medlemskap. Målet är att bygga en modern, responsiv och användarvänlig bokningsplattform som betjänar både medlemmar och administratörer på ett effektivt sätt.

2. Projektmål

- Utveckla ett modernt gymbokningssystem
- Låta användare registrera sig och logga in
- Låta medlemmar se och boka träningssessioner
- Låta administratörer hantera gymklasser och scheman
- Leverera en responsiv applikation för dator, surfplatta och mobil

3. MVP-omfattning

Användarfunktioner

- Registrera konto
- Logga in / logga ut
- Se kommande träningssessioner
- Boka en träningssession
- Avboka en bokning
- Visa profilinformation

Administratörsfunktioner

- Skapa träningssessioner
- Redigera träningssessioner
- Ta bort träningssessioner
- Hantera scheman

4. Agilt arbetssätt

Utvecklingsteamet använde agil metodik under hela projektet. **Trello** användes för sprintplanering och uppgiftshantering med veckovisa MVP-leveranser.

Arbetsflödeskolumner

- Eftersläp (Backlog)
- Att göra
- Pågår
- Testning

- Klar

Teamet levererade en körbar MVP-version varje vecka, vilket säkerställde kontinuerliga framsteg och tidig återkoppling.

5. Git-arbetsflöde

Git och GitHub användes för versionshantering med en funktionsgren-strategi. Alla färdiga funktioner slogs ihop med **main**-grenen via pull requests, vilket säkerställde kodgranskning före integration.

Grenexempel

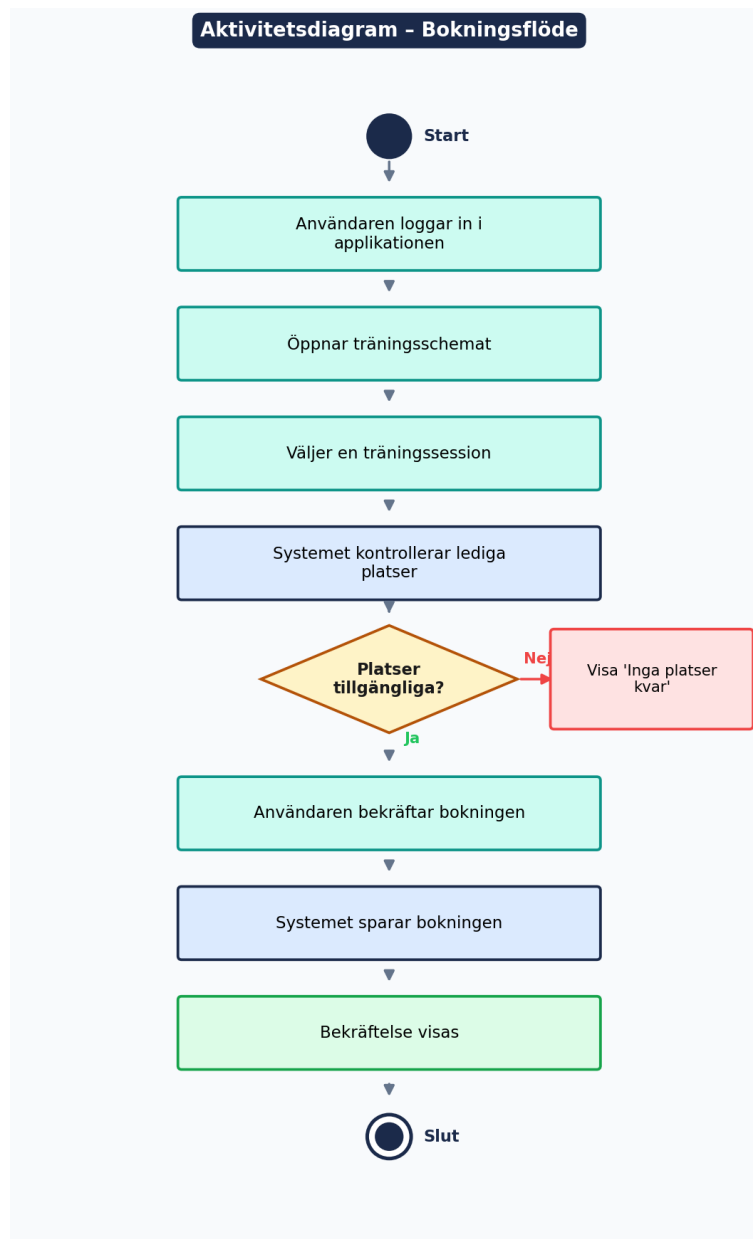
- feature/login
- feature/booking-system
- feature/admin-dashboard
- feature/profile-page

6. Användarberättelser

ID	Roll	Jag vill...	Så att...
US-01	Användare	skapa ett konto	jag kan använda bokningssystemet
US-02	Användare	logga in	jag kan hantera mina bokningar
US-03	Användare	logga ut	mitt konto förblir säkert
US-04	Medlem	se kommande gymklasser	jag kan välja ett lämpligt pass
US-05	Medlem	boka ett pass	jag kan reservera min plats
US-06	Medlem	avboka en bokning	jag kan frigöra min reserverade plats
US-07	Medlem	se mina kommande bokningar	jag kan hantera mitt schema
US-08	Administratör	skapa träningssessioner	medlemmar kan boka dem
US-09	Administratör	redigera träningssessioner	felaktig info kan uppdateras
US-10	Administratör	ta bort sessioner	inställda klasser tas bort från schemat

7. Aktivitetsdiagram – Bokningsflöde

Aktivitetsdiagrammet nedan modellerar det steg-för-steg-flöde en medlem följer för att boka ett gympass, inklusive systemets beslutspunkt som kontrollerar tillgängligheten av platser.



Figur 1 – Aktivitetsdiagram: Bokningsflöde för Medlem

Diagramgenomgång

Start: Flödet börjar när en registrerad medlem initierar en sessionsbokning.

Inloggning: Användaren autentiserar sig via ASP.NET-autentisering.

Öppna schema: Medlemmen navigerar till träningsschema-sidan.

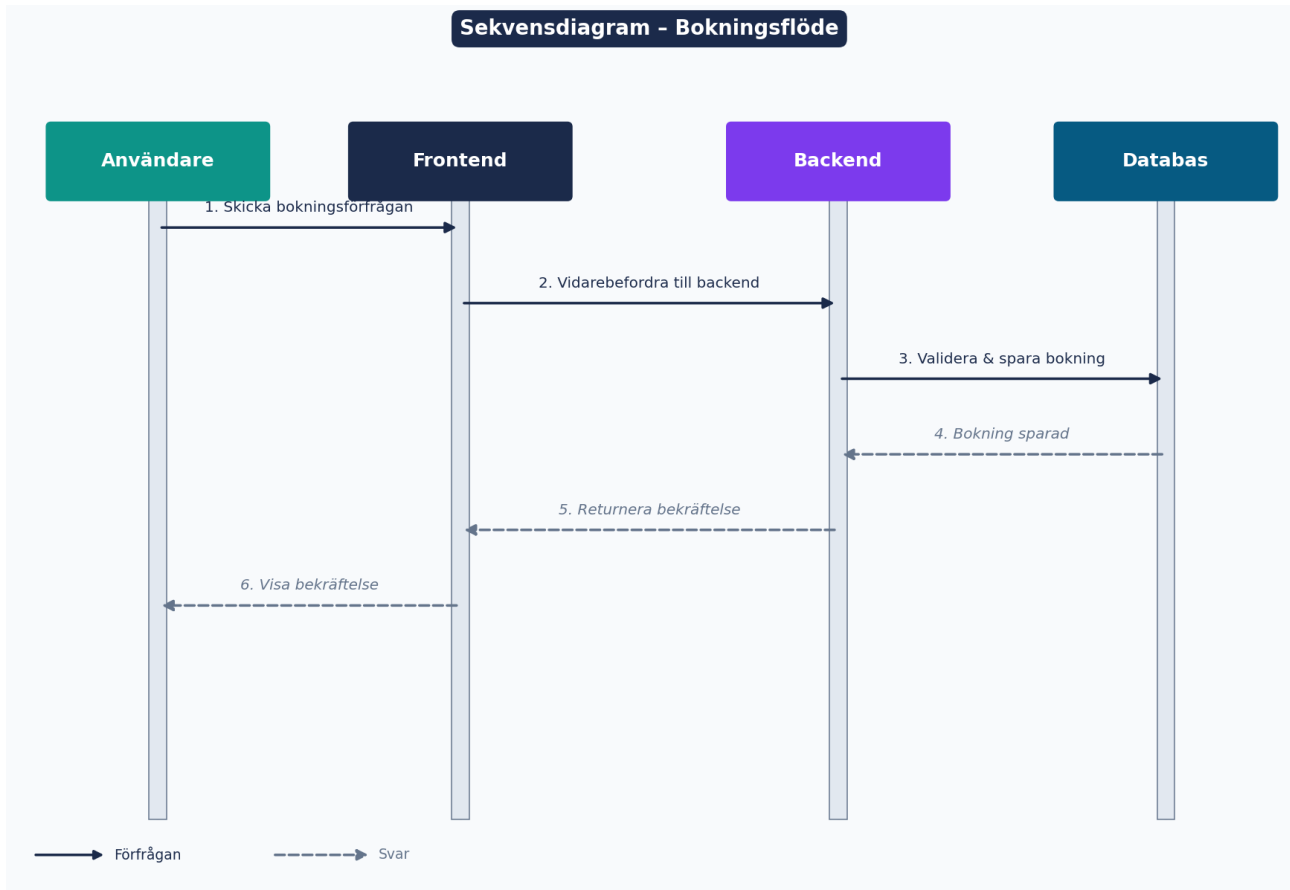
Välj session: Medlemmen väljer en önskad träningssession.

Tillgänglighetskontroll: Systemet kontrollerar återstående platser i databasen.

Beslut – Platser tillgängliga? JA → Användaren bekräftar → Systemet sparar bokningen → Bekräftelse visas → Slut. NEJ → Meddelandet 'Inga platser kvar' visas → Slut.

8. Sekvensdiagram – Bokningsflöde

Sekvensdiagrammet illustrerar det kronologiska meddelandeutbytet mellan **Användaren**, **Frontend** (React), **Backend** (ASP.NET Core Web API) och **Databasen** (SQLite) under en bokningsoperation.



Figur 2 – Sekvensdiagram: Bokningsflöde för Medlem

Beskrivning av meddelandeflöde

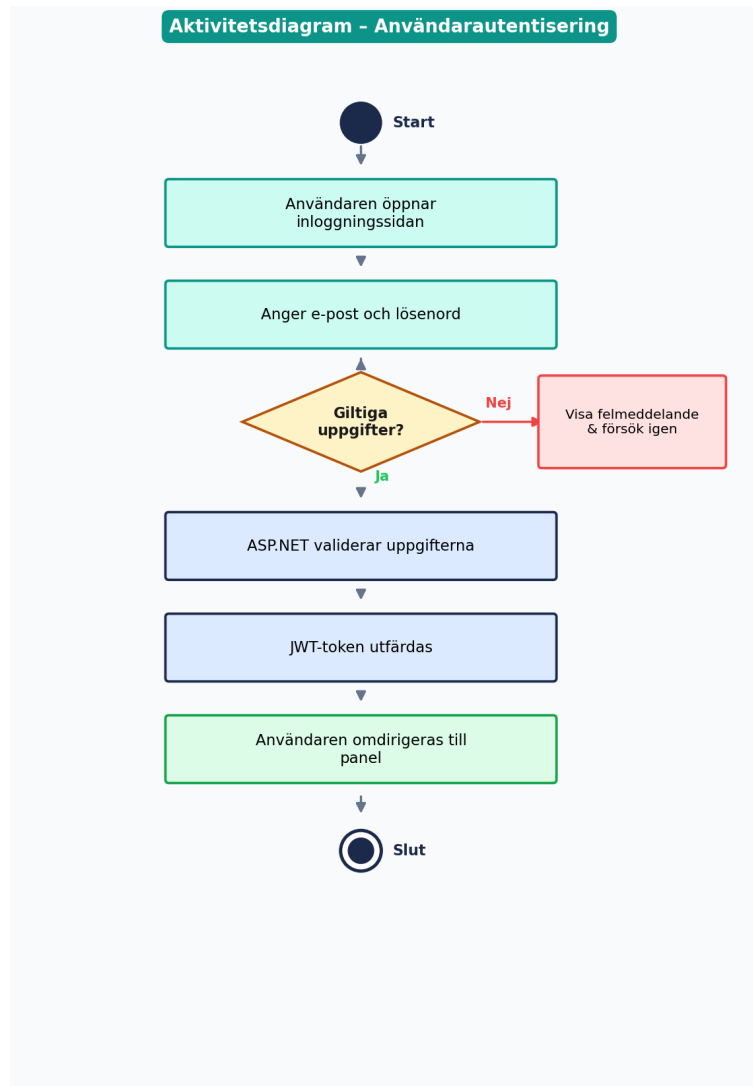
- Skicka bokningsförfrågan:** Användaren trycker på 'Boka' i gränssnittet.
- Vidarebefordra till backend:** React skickar ett API-anrop till ASP.NET Core.
- Validera & spara:** Backend kontrollerar platsantal och skriver till databasen.
- Bokning sparad:** Databasen bekräftar skrivningen (svar).
- Returnera bekräftelse:** Backend skickar framgångssvar till frontend.
- Visa bekräftelse:** Gränssnittet renderar ett framgångsmeddelande.

9. Användarautentiseringsmodul

Autentisering hanteras helt av **ASP.NET-autentisering**. Modulen stöder e-post/lösenords-registrering och inloggning, sessionspersistens och säker utloggning. En JWT-token utfärdas vid lyckad inloggning och används för alla efterföljande autentiserade förfrågningar.

9.1 Aktivitetsdiagram – Användarautentisering

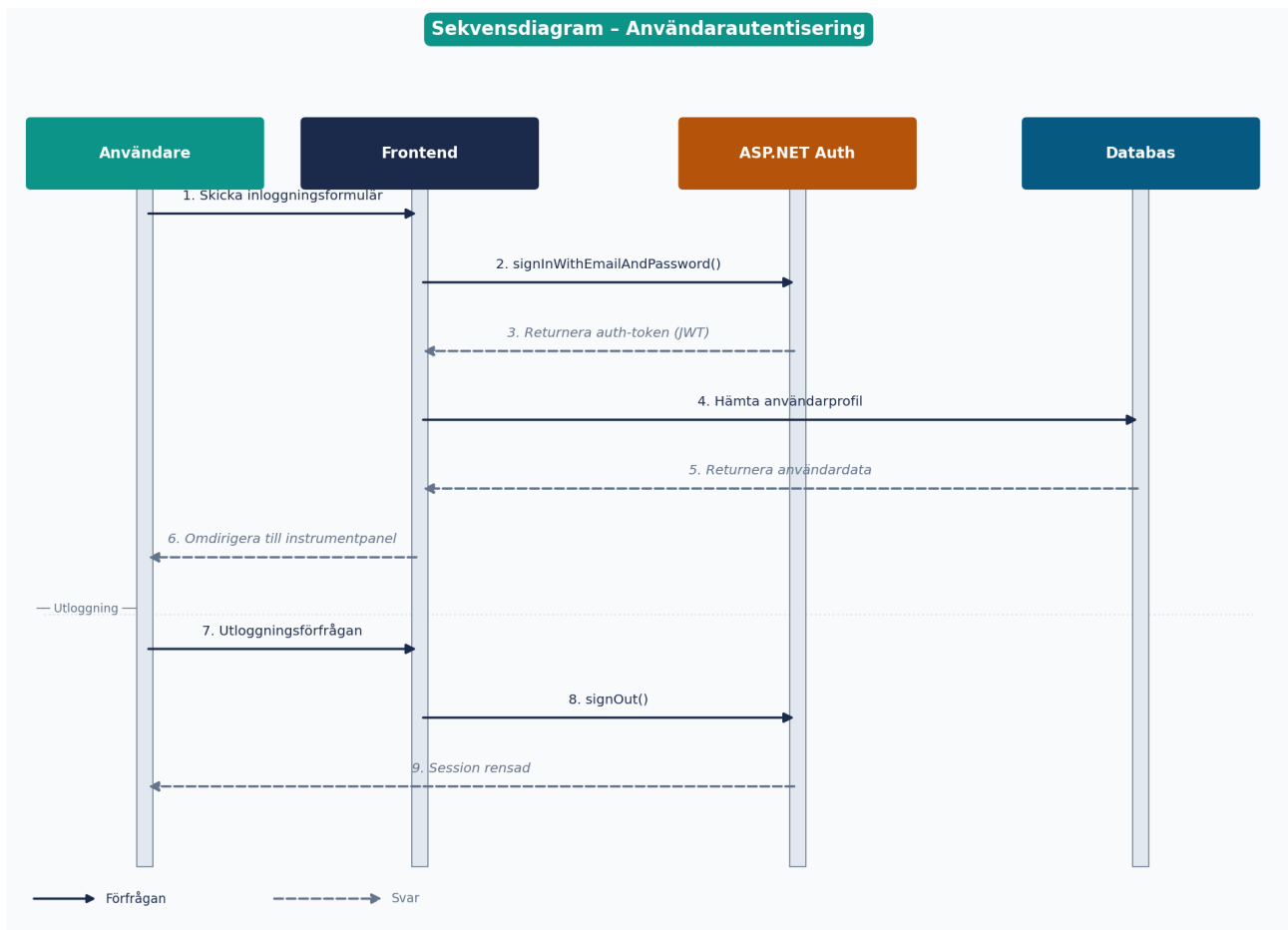
Diagrammet nedan modellerar inloggnings- och valideringsflödet, inklusive förgreningslogik för ogiltiga inloggningsuppgifter.



Figur 3 – Aktivitetsdiagram: Användarautentiseringsflöde

9.2 Sekvensdiagram – Användarautentisering

Detta diagram visar det fullständiga meddelandeutbytet från inloggningsformulärets inskickning via tokenutfärdande till omdirigering av instrumentpanelen, samt utloggningssekvensen.



Figur 4 – Sekvensdiagram: Användarautentiseringsflöde

Autentiseringsgenomgång

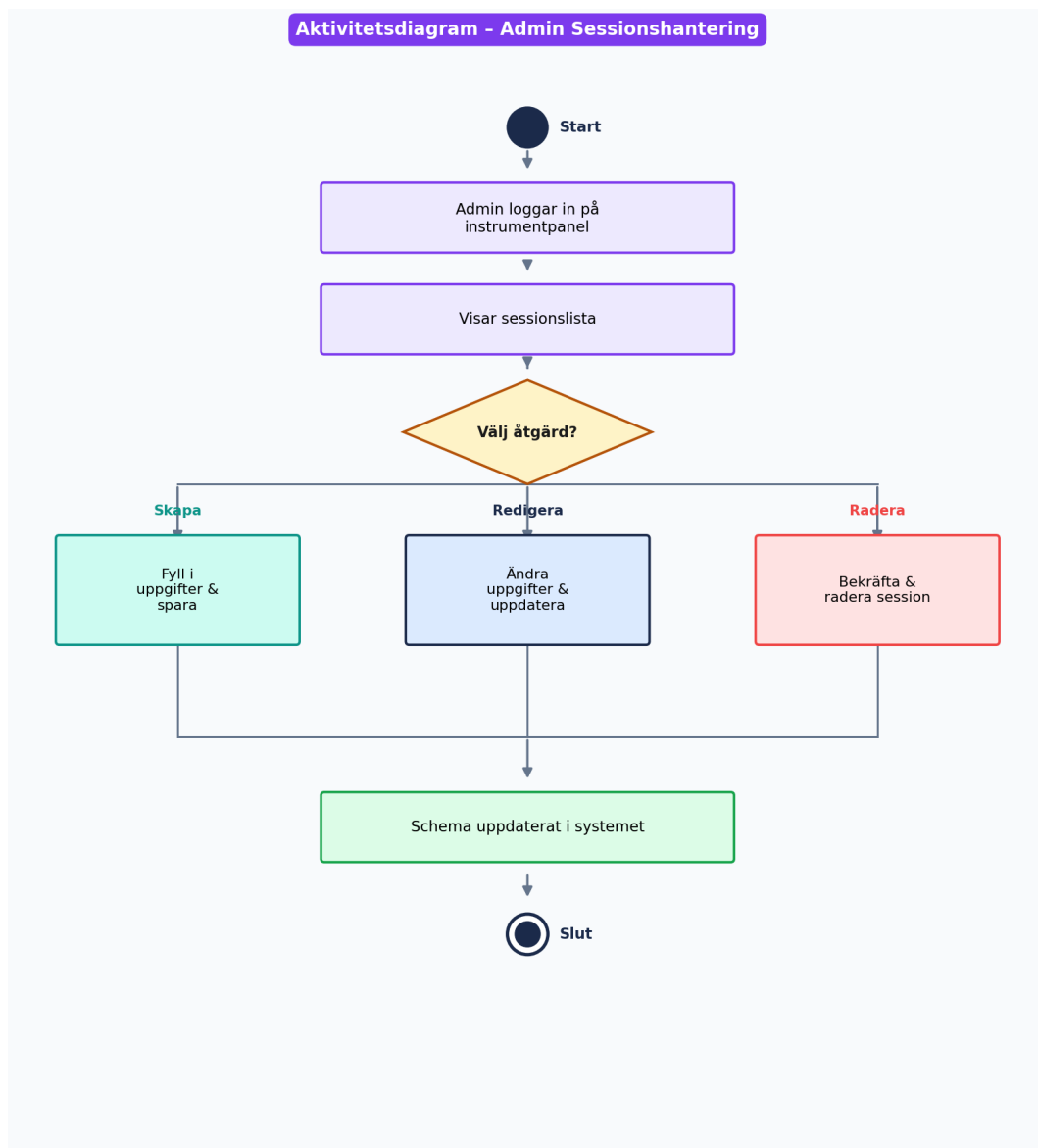
1. Användaren öppnar inloggningssidan och skickar in e-post/lösenord.
2. Frontend anropar `signInWithEmailAndPassword()` via ASP.NET SDK.
3. ASP.NET Auth validerar uppgifterna och returnerar en JWT-token.
4. Frontend hämtar användarprofildokumentet från databasen.
5. Användaren omdirigeras till medlemsinstrumentpanelen.
6. Vid utloggning anropas `signOut()`, vilket rensar sessionen.

10. Adminsessionshanteringsmodul

Adminmodulen ger autentiserade administratörer full kontroll över träningsessioner: skapa nya sessioner, redigera befintliga och ta bort inställda sessioner. Alla ändringar sparas i realtid i databasen.

10.1 Aktivitetsdiagram – Admin Sessionshantering

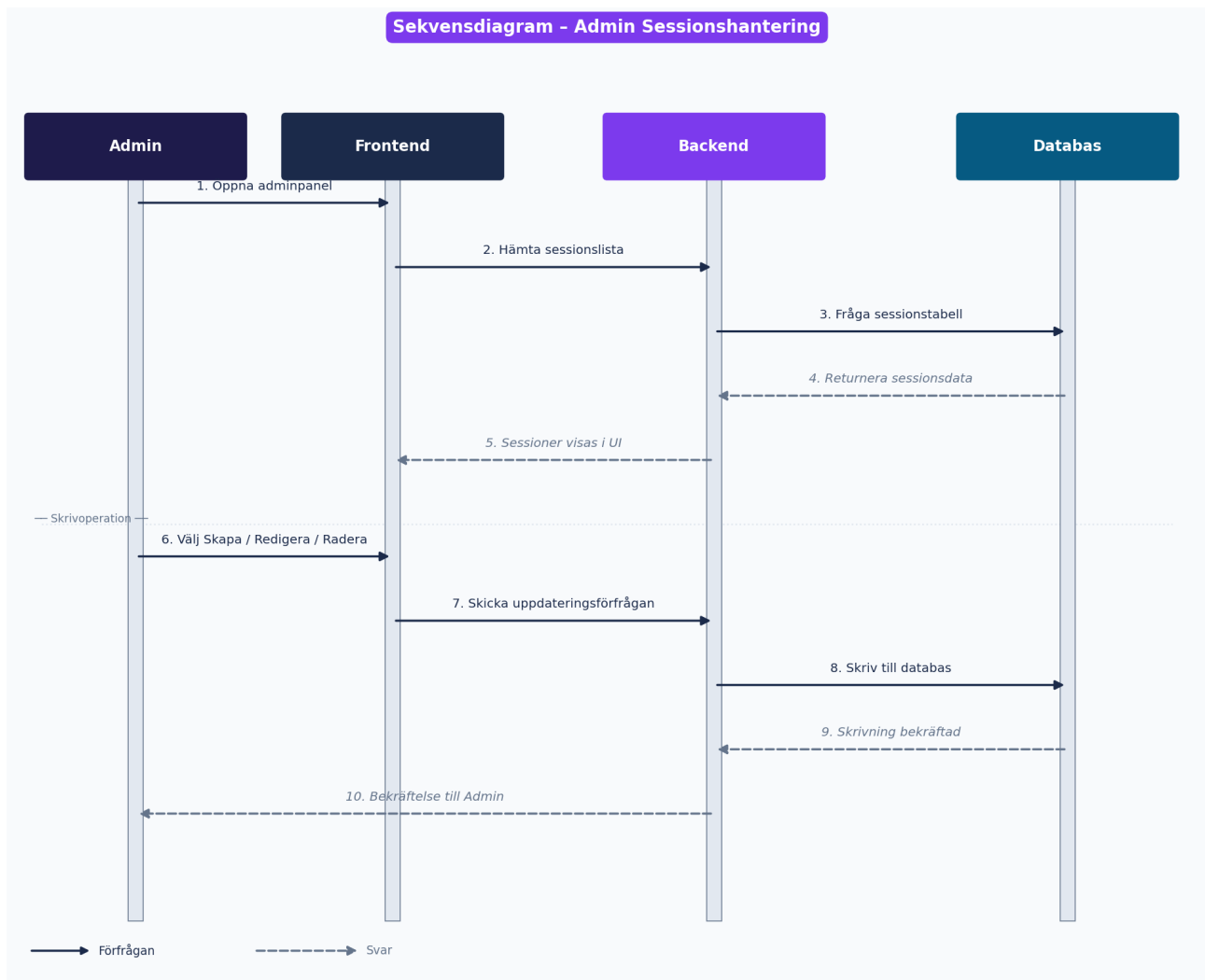
Det förgrenade aktivitetsdiagrammet nedan visar de tre vägarna en admin kan ta efter att ha sett sessionslistan: Skapa, Redigera eller Radera.



Figur 5 – Aktivitetsdiagram: Admin Sessionshantering

10.2 Sekvensdiagram – Admin Sessionshantering

Detta diagram fångar den fullständiga interaktionen mellan admin, frontend, backend och databasen – från inläsning av sessionslistan till bekräftelse av en skapa/redigera/radera-operation.



Figur 6 – Sekvensdiagram: Admin Sessionshantering

Adminflödesgenomgång

1. Admin öppnar adminpanelen (autentiserad rutt).
2. Frontend hämtar den aktuella sessionslistan via backend.
3. Sessioner renderas i adminens gränssnitt.
4. Admin väljer Skapa, Redigera eller Radera.
5. Uppdateringsförfrågan skickas till backend.
6. Backend skriver ändringen till sessionstabellen i databasen.
7. Databasen bekräftar skrivningen; admin får en framgångsavisering.

11. Teknisk stack

Lager	Teknik
Frontend	React
Backend	ASP.NET Core Web API
Databas	SQLite
ORM	Entity Framework Core
Autentisering	ASP.NET
API-dokumentation/Testning	Swagger
Versionshantering	Git + GitHub
Grenhantering	Funktionsgrenar + Pull Requests
Projekthantering	Trello
Design	Responsiv webbdesign
Utvecklingsverktyg	Visual Studio + VS Code
Programmeringsspråk	C#, JavaScript, HTML, CSS, React
Driftsättning / Hosting	GitHub

12. Responsiv design

Applikationen utformades för att fungera sömlöst på alla enhetstyper:

- Mobila enheter (iOS och Android via responsiv React-webbdesign)
- Surfplattor
- Webbläsare på dator (React-webb)

Responsiva layouter implementerades för att förbättra användbarheten och bibehålla en konsekvent upplevelse på alla skärmstorlekar.

13. Utmaningar

Utmaning	Lösning
Konfigurationsproblem för miljövariabler	Förbättrade initialiseringsskript
Krascher i produktionsbygge i Swagger	Noggrann testning & reservkonfigurationer
Hantering av autentiseringspersistens	Inbyggda persistensinställningar aktiverades
Offline-tillstånd och API-fel	Felhantering och reservkonfigurationer lades till

14. Veckovis MVP-progression

Sprint	Tema	Viktiga leveranser
Vecka 1	Grund	• Projektkonfiguration och uppbyggnad • ASP.NET Core Web API-projektkonfiguration • Autentiseringssystem (registrering/inloggning/utloggning)
Vecka 2	Kärnbokning	• Implementering av bokningssystem • Sessionslistning och filtrering • Användarprofilfunktioner
Vecka 3	Admin & Finslipning	• Adminpanel • Skapa / redigera / ta bort sessioner • Förbättringar av responsiv layout
Vecka 4	Leverans	• Heltäckande testning och felrättning • Slutlig driftsättning via GitHub • Projektdokumentation

15. Reflektioner

Projektet fördjupade teamets förståelse för modern fullstack-mobil/webbutveckling. Viktiga lärdomar inkluderade:

- Agil utveckling och sprinthantering med Trello
- Git-arbetsflöde: funktionsgrenar, pull requests och kodgranskning
- ASP.NET Core Web API, Entity Framework Core och SQLite: autentisering och realtidsuppdateringar
- Plattformsoberoende responsiv design för mobil och webb
- Felsökning och testning av produktionsbyggen i Swagger
- MVP-planering och iterativ funktionsprioritering

Teamet förstärkte också vikten av:

- Kontinuerlig testning under hela utvecklingen
- Noggrann dokumentation
- Disciplinerad versionshantering
- Inkrementell MVP-leverans för snabba återkopplingscykler

16. Slutsats

Projektet levererade framgångsrikt ett fungerande MVP-bokningssystem för Core Gym Club AB. Applikationen byggdes med agila metoder, Git-arbetsflöden och veckovisa MVP-leveranser i enlighet med uppgiftskraven.

Medlemmar kan:

- Registrera sig och logga in säkert
- Se alla tillgängliga gympass
- Boka och avboka träningssessioner
- Visa och hantera sitt kommande schema

Administratörer kan:

- Skapa, redigera och ta bort träningssessioner
- Uppdatera och hantera hela träningsschemat